

[E4] Atividade Prática

Caça ao Defeito: Diagnóstico de Falha no Caminho de Dados

Nome: _____ RA: _____

1. Definições

O Projeto PRISMA é uma iniciativa para tornar mais visual e interativo o processo de estudo e desenvolvimento de componentes para um processador, fornecendo uma base pronta para que testes e experimentos possam ser realizados na arquitetura RISC-V. A visualização dos dados que transitam em conexões e a análise de componentes e portas lógicas que constituem o processador é possível por meio do uso do simulador visual de circuitos digitais Logisim-Evolution.

A implementação disponível no projeto a ser utilizada nessa atividade é:

- Caminho de Dados Monociclo com defeito (GitHub)

Nessa arquitetura, toda instrução é executada em um único ciclo de *clock*, e as unidades funcionais (banco de registradores, ULA, memória de dados e multiplexadores) são compartilhadas entre todas as instruções. Cada instrução percorre um subconjunto diferente dessas unidades, e é essa diferença que torna o diagnóstico possível: um componente defeituoso corrompe apenas as instruções que passam por ele.

Nessa cópia do caminho de dados foi introduzido um único defeito, a respeito do qual se sabe apenas o seguinte:

- É combinacional: não depende do estado anterior nem do número de ciclos já executados. A mesma instrução, na mesma posição e sobre os mesmos operandos, produz sempre o mesmo resultado errado.
- Não é sinalizado: o processador não acusa falha, não há mensagem de erro nem indicação de estado inválido. A evidência é sempre indireta, um valor diferente do previsto, uma instrução que não deveria ter executado, ou uma execução que termina antes ou depois do esperado.
- Não afeta todas as instruções: existem instruções que executam corretamente. A fronteira entre as que falham e as que não falham é a principal pista da atividade, e ela não é necessariamente a fronteira entre formatos, pode separar instruções de um mesmo formato conforme os operandos ou os valores envolvidos.

Importante: o defeito não é identificável por inspeção visual do circuito. Procurá-lo abrindo os componentes e comparando fios com o material de referência é o caminho mais lento e não é o que se avalia aqui. Ele é localizável por experimentação, escrevendo programas cujo resultado você sabe prever e observando onde a previsão falha.

As instruções suportadas pelo Projeto PRISMA estão descritas na Tabela 1, e todas seguem as especificações da arquitetura RISC-V.

Tabela 1 - Instruções suportadas

Aritmética/Lógicas (AL)	AL com imediato	Memória	Desvios
add	addi	lb	beq
sub	xori	lh	bne
xor	ori	lw	blt

or	andi	lbu	bge
and	slli	lhu	bltu
sll	srli	sb	bgeu
srl	srai	sh	jal
sra	slti	sw	jalr
	sltiu		
	lui		
	auipc		

O caminho de dados é montado a partir dos componentes da Tabela 2, e qualquer um deles pode ser a origem da falha. Essa lista é o espaço de hipóteses da atividade: ao final, a explicação deve apontar um componente, o sinal ou fio interno responsável e o motivo pelo qual ele produz exatamente a divergência observada, e não outra.

Tabela 2 - Componentes do caminho de dados monociclo

Componente	Função
ULA	Soma, subtração, lógicas, deslocamentos e comparações.
ucULA	Traduz ALUOp mais funct3 e funct7 no código da operação da ULA.
ucPrincipal	Gera os sinais de controle a partir do opcode.
ucDesvio	Decide se o desvio condicional é tomado.

ucEndereçamento	Define tamanho e alinhamento dos acessos à memória.
bancoRegistradores	Banco com os 32 registradores de 32 bits.
decodificador	Separa os campos da instrução e monta o imediato.
extensores	Extensão de sinal e de zero, usados dentro de outros componentes.
memInstrucoes	Memória de instruções (ROM).
memDados	Memória de dados (RAM).
zero	Sinalizador de resultado nulo da ULA.

Conceitos que serão utilizados na atividade:

- **Resultado previsto:** o efeito que uma instrução deve produzir sobre o banco de registradores, a memória de dados ou o PC, determinado antes da simulação, a partir da especificação da ISA. Sem previsão anterior não há divergência: há apenas um valor cuja correção se descobre depois de vê-lo.
- **Teste discriminante:** programa escrito para separar duas ou mais hipóteses, produzindo resultados diferentes conforme qual delas seja verdadeira. Um teste que falha do mesmo jeito sob todas as hipóteses não elimina nenhuma e não deve ser incluído no relatório como evidência.
- **Cadeia causal:** a sequência que liga o defeito ao sintoma, ou seja, o fio ou porta alterado, o sinal que ele produz de forma incorreta, a unidade que consome esse sinal e o valor errado que chega ao destino final.

2. Atividade

Localize, explique e corrija o defeito introduzido no caminho de dados Monociclo do Projeto PRISMA. O percurso é sempre o mesmo: escrever um programa de verificação e prever seus resultados, formular hipóteses a partir das divergências e escrevê-las como testes que as discriminem, e confirmar a origem da falha, descrever a cadeia causal e corrigir o circuito.

O formato de cada instrução e a posição dos campos opcode, funct3 e funct7 estão no cartão de referência RISC_V_CARD.pdf, e a documentação de cada componente está no diretório componentes do projeto, no GitHub.

2.1 Preparação

1. Abra o componente Caminho de Dados Monociclo com defeito no Logisim-Evolution 4.1.0.
2. Guarde uma cópia intacta do arquivo antes de qualquer alteração. A correção só é avaliável se houver como comparar o circuito corrigido com o circuito recebido.
3. Escreva o programa em uma imagem no formato v2.0 raw e carregue-a com o botão direito na memória de instruções → Load Image.
4. Antes de cada execução, zere o banco de registradores, a memória de dados e o PC com Simulate → Reset Simulation (Ctrl+R). Um estado residual de uma execução anterior produz divergências que não têm relação com o defeito.
5. Faça a depuração avançando a execução manualmente, um ciclo por vez (Ctrl+T), conferindo o efeito de cada instrução antes de prosseguir. Encerre o programa com a palavra ffffffff.
6. Se a execução parar antes do fim do programa, ou não alcançar a palavra ffffffff, registre isso como divergência: é evidência tão legítima quanto um valor errado em registrador. Reinicie com Ctrl+R e prossiga a investigação a partir dela.

2.2 Programa de verificação e previsão

Uma instrução testada uma única vez, com um único conjunto de operandos, só autoriza a conclusão de que ela funciona **naquele caso**. Como o defeito pode depender dos valores envolvidos, e não apenas da instrução, cada uma deve aparecer mais de uma vez, com operandos diferentes entre si.

O programa deve conter, no mínimo:

- **Tipo R** — pelo menos duas operações distintas, sobre pares de registradores distintos, com valores em que a soma, o or e o xor produzam resultados diferentes. Operar sobre 0 e 1 faz muitas operações coincidirem no mesmo resultado.
- **Tipo I aritmético** — imediatos de grandezas diferentes, incluindo ao menos um negativo e um com o bit mais significativo do campo ativo.
- **Tipo I de carga e tipo S** — acessos em posições de memória diferentes e em larguras diferentes, cobrindo byte, half e word.
- **Tipo B** — pelo menos três desvios, com **distâncias de salto diferentes entre si**, e para cada um deles um caso em que a condição é verdadeira e um em que é falsa.
- **Tipo U** — pelo menos dois imediatos distintos.
- **Tipo J** — pelo menos dois saltos, com **distâncias diferentes entre si**.
- **Registradores** de índices variados, altos e baixos, e não apenas x1 e x2.

Preencha a Tabela 3 antes de simular, com o efeito previsto de cada instrução, e só então execute o programa e registre o resultado obtido. Acrescente linhas conforme o tamanho do seu programa. Se todas as divergências estiverem concentradas em um mesmo tipo de instrução, descreva também o que difere entre as ocorrências desse tipo que falham e as que não falham.

#	Endereço	Instrução (assembly)	Palavra (hex)	Efeito previsto	Resultado obtido	Diverg e?
---	----------	-------------------------	------------------	--------------------	---------------------	--------------

1						
2						
3						
4						
5						
6						
7						
8						
9						
10						
11						
12						
13						
14						
15						

Tabela 3 - Programa de verificação: previsão e resultado

Importante: entregue a tabela de previsões ao docente para visto antes de qualquer simulação. Previsão registrada depois da execução não é previsão: é anotação do que o circuito fez, e não permite distinguir o que você sabia do que o simulador mostrou.

Ao final da execução, o conjunto de linhas com divergência já delimita o problema. Descreva em uma frase o que essas instruções têm em comum e, igualmente importante, o que as instruções sem divergência têm em comum.

2.3 Hipóteses e testes discriminantes

A partir das divergências, formule hipóteses sobre a origem da falha e escreva, para cada uma, um programa que a discrimine das demais. Cada teste deve variar uma característica por vez: mesma operação com registradores diferentes, mesmo registrador com operações diferentes, mesmo valor obtido por caminhos diferentes.

Registre o processo na Tabela 4. Uma hipótese só é eliminada quando existe um resultado observado incompatível com ela.

Hipótese	Teste (programa e instrução-chave)	Resultado se a hipótese for verdadeira	Resultado obtido	Conclusão
				Eliminada / Sustentada
				Eliminada / Sustentada
				Eliminada / Sustentada
				Eliminada / Sustentada

Tabela 4 - Hipóteses e testes discriminantes

Antes de escrever cada teste, responda: qual resultado eu esperaria se essa hipótese fosse falsa? Um teste cujo resultado é o mesmo nos dois casos não elimina nada, e testes redundantes, que repetem uma discriminação já feita, não acrescentam evidência ao relatório.

2.4 Confirmação, cadeia causal e correção

Isolada a variável responsável, confirme a origem da falha por observação direta, e não apenas por eliminação: execute novamente o teste que a revela e inspecione os fios internos do componente suspeito durante a simulação, descendo ao interior dele pelo menu de contexto (botão direito sobre o componente, opção de visualizá-lo) e comparando o valor de cada entrada e saída com o valor que a especificação exige naquele ciclo.

Descreva então a cadeia causal completa, na ordem: fio ou porta alterado → sinal incorreto que ele produz → unidade que consome esse sinal → valor errado que chega ao registrador, à memória ou ao PC → divergência registrada na Tabela 3.

Corrija o circuito no simulador e reexecute todos os programas usados até aqui, incluindo os testes que já passavam. Uma correção que conserta a instrução investigada e quebra outra não é uma correção.

Programa	Divergências antes da correção	Divergências após a correção
Programa de verificação		
Teste 1		
Teste 2		
Teste 3		

Tabela 5 - Reexecução após a correção

Observação: se algum programa continuar divergindo após a correção, a hipótese sustentada na seção 2.3 explicava apenas parte do sintoma. Volte à Tabela 4, registre o que o novo resultado elimina e retome a investigação. O percurso, inclusive as hipóteses descartadas, faz parte da entrega.

3. Relatório de Diagnóstico

Prepare um relatório que apresente:

1. Os programas utilizados, o de verificação e os de teste, em assembly e em linguagem de máquina, com a Tabela 3 preenchida: resultado previsto, resultado obtido e divergências.
2. A justificativa de cada teste da Tabela 4: qual hipótese ele discrimina, qual resultado se esperava sob cada hipótese e qual hipótese o resultado obtido eliminou.
3. A cadeia causal entre o defeito no circuito e a divergência observada, nomeando o componente, o fio ou porta responsável e o sinal incorreto que dele resulta.
4. A evidência da correção: a alteração feita no circuito e a Tabela 5, mostrando que todos os programas passaram a produzir os resultados previstos na seção 2.2.